Politecnico di Torino

Master of Science (M.Sc.) in Computer Engineering

Academic Year 2025/2026

Graduation Session March 2026

Design and Implementation of a Cloud-Native Reactive Architecture

The Smart Travel Platform Case Study

Supervisor

Prof. Giovanni Malnati

Candidate

Federico Palazzi

Abstract

The rapid evolution of enterprise web applications has driven a paradigm shift from monolithic architectures to cloud-native *microservices* solutions. This transition is motivated by the need for greater scalability, resilience, and agility in the software development life cycle. This thesis presents the design, development and deployment of "*Smart Travel*", a comprehensive e-commerce platform dedicated to the sale of travel packages. The project serves as a proof of concept (PoC) developed within an industrial internship context, aiming to analyze the complexities and benefits of a modern and distributed software architecture.

The primary objective of this work is not merely the implementation of functional requirements (such as catalog management and order processing), but rather the critical evaluation of competing technologies and the automation of the release process. A central element of this research is the comparative analysis between two leading Java ecosystems: the established *Spring Boot* framework and the cloud-native *Quarkus* framework. Specifically, the project adopts a fully reactive approach for both of them, utilizing *Spring WebFlux* and *Quarkus Reactive* capabilities.

From an architectural perspective, the Smart Travel system is designed to ensure loose coupling and high availability. The backend adopts a microservices pattern supported by *RabbitMQ*, a message broker that facilitates asynchronous *event-driven* communication among services. Data persistence is managed through *MongoDB*, a NoSQL database chosen for its schema flexibility, which is particularly advantageous for handling diverse and evolving travel package structures compared to rigid relational models. On the client side, the user interface is delivered via a *Single Page Application (SPA)* developed with *Angular* and leveraging *RxJS* for reactive state management, ensuring a responsive and fluid user experience.

A significant portion of this thesis is dedicated to the "industrialization" of the software, bridging the gap between development and operations (*DevOps*). The project moves beyond local execution by implementing a robust *Continuous Integration and Continuous Delivery (CI/CD)* pipeline using *Jenkins*. This pipeline automates the entire delivery chain, including code checkout, compilation, container image creation and pushing to a private registry. The final deployment target is *OKD* (the community distribution of *Kubernetes* that powers *Red Hat OpenShift*). The thesis details the orchestration of containerized applications within the OKD cluster, managing critical aspects such as service discovery, route exposure and configuration management via `ConfigMaps` and `Secrets`.

In conclusion, the Smart Travel project demonstrates the viability and robustness of a heterogeneous, fully reactive microservices architecture. It successfully integrates different modern technologies into a cohesive ecosystem, proving that the adoption

of cloud-native principles, while introducing complexity in orchestration and consistency management, offers superior long-term benefits in terms of maintainability and scalability. The work provides a practical blueprint for modernizing legacy Java applications, offering concrete evidence of how the synergy between Reactive frameworks, Event-Driven patterns, and automated DevOps practices defines the state of the art in contemporary software engineering.

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Table of Contents

List of Figures VII

Glossary IX

1 Introduction 1
- 1.1 Context . . . 1
- 1.2 Goal . . . 2
  - 1.2.1 Project description . . . 2
  - 1.2.2 Expected Results . . . 3
- 1.3 Thesis structure . . . 4

2 Background and Technologies 5
- 2.1 Microservices Architecture . . . 5
  - 2.1.1 Principles of Microservices . . . 5
  - 2.1.2 Benefits . . . 7
  - 2.1.3 Challenges and Limitations . . . 7
- 2.2 Imperative vs. Reactive paradigm . . . 8
  - 2.2.1 The Imperative Model . . . 8
  - 2.2.2 The Reactive Model . . . 9
- 2.3 Event-Driven Architectures . . . 10
  - 2.3.1 Synchronous vs. Asynchronous Communication . . . 10
  - 2.3.2 The Role of Message Brokers . . . 10
  - 2.3.3 Benefits and Challenges . . . 11
- 2.4 Backend Technologies . . . 12
  - 2.4.1 Spring Boot: The Industry Standard . . . 12
  - 2.4.2 Quarkus: The Cloud-Native Framework . . . 13
  - 2.4.3 The Rationale for Comparison . . . 13
- 2.5 Frontend Technologies . . . 13
  - 2.5.1 Single Page Application Architecture . . . 14
  - 2.5.2 The Angular Framework . . . 15
  - 2.5.3 Frontend Reactive Programming . . . 16

2.6 Database Technologies 16
2.6.1 NoSQL Concepts and Classifications 16
2.6.2 MongoDB Architecture and Flexible Schema 17
2.6.3 Scalability and High Availability 18
2.7 API Paradigms: REST vs. GraphQL 18
2.7.1 REST (Representational State Transfer) 19
2.7.2 GraphQL: The Client-Driven Alternative 19
2.7.3 Choosing the Right Paradigm 20

3 System Architecture 21

4 Application Design and Implementation 22

5 Infrastructure Automation and Cloud Deployment 23

6 Conclusion 24

Bibliography 25

List of Figures

2.1 Monolithic vs. Microservices architecture 6
2.2 Thread-per-Request model in an imperative application. 8
2.3 Reactive model in a non-blocking application. 9
2.4 Event-Driven Architecture 11
2.5 Angular Architecture 15
2.6 NoSQL Database Types 17
2.7 REST vs. GraphQL 20

Glossary

ACID

Atomicity, Consistency, Isolation, Durability

AJAX

Asynchronous JavaScript and XML

API

Application Programming Interface

BSON

Binary JSON

CI/CD

Continuous Integration / Continuous Delivery

DBMS

Database Management System

DI

Dependency Injection

DOM

Document Object Model

DX

Developer Experience

EDA

Event Driven Architecture

JSON

JavaScript Object Notation

MPA

Multi-Page Application

NoSQL

Not Only SQL

ORM

Object-Relational Mapping

PoC

Proof of Concept

Pub/Sub

Publish/Subscribe

RDBMS

Relational Database Management System

RxJS

Reactive Extensions for JavaScript

SEO

Search Engine Optimization

SPA

Single Page Application

UI

User Interface

Chapter 1

Introduction

1.1 Context

The rapid evolution of enterprise web applications over the last decade has driven a significant paradigm shift in software engineering, moving away from traditional monolithic architectures towards cloud-native microservices solutions.

Historically, monolithic applications were the industry standard and they integrated user interface, business logic, and data access layers into a single deployable unit. They offered simplicity during the early stages of development, straightforward local testing, and centralized debugging. However, as applications grow in size, complexity, and user base, the inherent limitations of the monolithic approach become increasingly apparent [1].

A monolithic design relies on tightly coupled components. Consequently, a minor update, a new feature addition, or a simple bug fix requires the entire massive codebase to be recompiled, tested, and redeployed. This rigid structure often leads to development bottlenecks and significantly increases the risk of system-wide failures during deployments. Furthermore, scaling a monolith is inherently inefficient; if a single specific component (such as the payment processing module) experiences a surge in traffic, the entire monolithic application must be scaled horizontally, leading to unnecessary resource consumption and inflated operational costs.

To overcome these scalability and maintainability challenges, the industry has widely adopted the microservices architecture [1]. This modern paradigm decomposes a large application into a suite of small, autonomous, and loosely coupled services. Each microservice is organized around a specific business capability, encapsulates its own data, and communicates with other services through lightweight, well-defined application programming interfaces (APIs) or asynchronous message brokers. This decentralized approach offers superior modularity, improved fault isolation - the

failure of one service does not inevitably crash the entire system - and the agility to deploy and scale individual services independently based on their specific demand. However, the transition to microservices is not without its own set of complexities. It introduces challenges such as increased operational overhead, the need for sophisticated monitoring and logging solutions, and the complexity of managing distributed transactions and data consistency across services. Despite these challenges, the microservices architecture has become the de facto standard for building scalable, resilient, and maintainable enterprise applications in the cloud era.

1.2 Goal

The primary objective of this thesis is to explore, apply, and critically evaluate modern cloud-native technologies and microservices architectures. To achieve this, the "*Smart Travel*" platform was designed and developed as a proof of concept (PoC). Rather than simply focusing on the implementation of functional business requirements, this project serves primarily as a practical testbed to analyze the complexities, challenges, and benefits of building a modern, distributed software system. The project was conducted during an internship at a software company, situated directly within the industry's broader context of architectural modernization.

1.2.1 Project description

The Smart Travel application was designed as a comprehensive e-commerce platform dedicated to the sale of travel packages. The system manages the entire customer journey, from the public consultation of a catalog containing flights, accommodations, and organized activities, to the execution of the purchasing process, order history management, and the automated dispatch of email notifications.
A central technical goal of this research is the critical evaluation and comparative analysis of two leading Java ecosystems used to build the platform's microservices: the established *Spring Boot* framework and the newer cloud-native *Quarkus* framework. Specifically, the project adopts a fully reactive programming paradigm for both frameworks, utilizing *Spring WebFlux* and *Quarkus Reactive* capabilities. This enables non-blocking, asynchronous execution, allowing the system to handle a massive number of concurrent requests with a highly optimized `thread pool`.
Furthermore, this thesis aims to explore the "industrialization" of software through the adoption of *DevOps* practices. The objective is to move beyond local execution by implementing a robust *Continuous Integration and Continuous Delivery* (CI/CD) pipeline. By automating the entire delivery chain - from code checkout and compilation to container image creation and pushing to a private registry - the project demonstrates how to bridge the gap between development and operations. Finally, the thesis details the deployment and orchestration of these containerized

microservices within an *OKD* cluster (the community distribution of *Red Hat OpenShift*), managing critical operational aspects such as service discovery, route exposure, and configuration management.

1.2.2 Expected Results

The execution of this thesis work is expected to produce both concrete technical artifacts and valuable analytical insights. The expected results can be categorized into three main areas:

1. From a software engineering perspective, the project aims to deliver a fully functional PoC of the e-commerce platform. This includes the successful implementation of the frontend interface and the backend microservices, validating the practical application of asynchronous communication via RabbitMQ and flexible data management using MongoDB. The resulting system is expected to demonstrate the resilience and scalability inherent in a properly designed cloud-native architecture.

2. From an analytical standpoint, the research provides a qualitative comparative evaluation between Spring Boot and Quarkus within a reactive programming context. Given the scope of the internship, the comparison focuses heavily on the *Developer Experience* (DX) rather than quantitative runtime metrics. The expected outcome is a clear assessment of the differences in syntax, configuration management, boilerplate code reduction, and the overall ease of implementing reactive paradigms in both frameworks. This comparison will offer valuable insights into the practical development lifecycle, ecosystem maturity, and the learning curve associated with adopting Quarkus compared to the traditional Spring environment.

3. From an operational perspective, the expected result is the successful demonstration of a fully automated DevOps lifecycle. By delivering a working CI/CD pipeline via Jenkins that culminates in a live deployment on an OKD cluster, the project will prove the viability and necessity of infrastructure automation. This includes the ability to manage containerized applications at scale, handle configuration and secrets management effectively, and ensure seamless service discovery and routing in a production-like environment.

1.3 Thesis structure

The thesis is structured as follows:

- **Chapter 2: Background and Technologies** - Provides an overview of the core concepts and technologies underpinning the project, including microservices and event-driven architectures, imperative and reactive programming paradigms, backend and frontend frameworks (Spring Boot, Quarkus, Angular), NoSQL databases (MongoDB), and API communication models (REST vs. GraphQL).
- **Chapter 3: System Architecture** - Details the architectural decisions, technology stack, and design patterns used in the Smart Travel project. It covers the microservices decomposition, the event-driven communication via message brokers, and the frontend design.
- **Chapter 4: Application Design and Implementation** - Offers a detailed description of the development process. This includes data modeling, API contract definitions, and explanations of key software components, accompanied by code snippets comparing the Spring Boot and Quarkus implementations.
- **Chapter 5: Infrastructure Automation and Cloud Deployment** - Describes the different steps of the CI/CD pipeline, Jenkins automation configurations, and the final orchestration of containerized applications within the OKD cluster environment.
- **Chapter 6: Conclusion** - Summarizes the project findings, evaluates the results of the technology comparison, discusses the limitations encountered during development, and outlines future work directions for the platform.

Chapter 2

Background and Technologies

This chapter provides a comprehensive overview of the core concepts and technologies underpinning the project. It begins by exploring the architectural shift from traditional monolithic systems to decentralized microservices, then moving to the main differences between imperative and reactive programming models, highlighting how the latter addresses the high-concurrency demands of modern cloud infrastructure.
Furthermore, it examines the project's technological stack and the different communication paradigms adopted to allow the system's components to interact with each other.

2.1 Microservices Architecture

The evolution of cloud computing has fundamentally altered how enterprise applications are designed, developed, and maintained. While monolithic architectures served as the industry standard for decades, they present significant scaling and maintenance limitations as systems grow in complexity. To address these bottlenecks, the software industry has increasingly adopted the microservices architecture, a paradigm that decomposes large, rigid applications into a suite of smaller, independent services [2].

2.1.1 Principles of Microservices

Unlike a monolithic application that shares a single massive codebase and a centralized database, a microservices architecture is built upon the principle of decentralization and loose coupling. Each microservice is designed around a specific

business capability or "bounded context" [3]. For example, in an e-commerce platform like Smart Travel, the catalog management and the order processing capabilities are handled by distinct autonomous services.
The foundational principles of this architecture include:

- **Autonomy and Independent Deployment:** Each service is a self-contained unit. It encapsulates its own code, dependencies, and typically its own data store. This allows teams to update, deploy, and scale a single service without needing to recompile or redeploy the entire system.

- **Lightweight Communication:** Because microservices operate as independent processes, they must communicate over a network to fulfill complex business transactions. This is achieved using well-defined *Application Programming Interfaces* (APIs), typically relying on synchronous protocols like HTTP or asynchronous message delivery mechanisms.

- **Decentralized Data Management:** Rather than relying on a single shared database schema, each microservice manages its own database, choosing the DBMS technology best suited to its specific data access patterns.

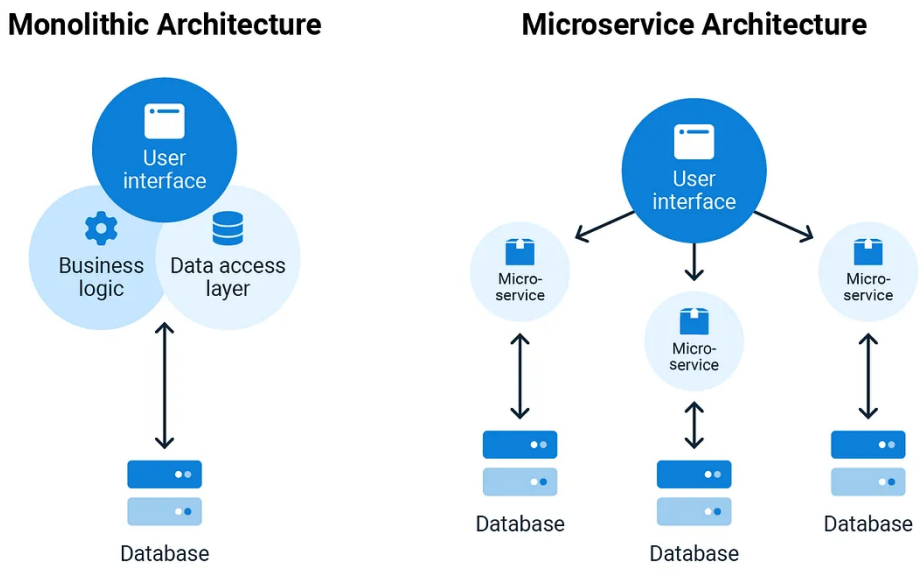

Figure 2.1: Monolithic vs. Microservices architecture. *Source: [4]*

2.1.2 Benefits

The shift from monolithic to microservices architecture provides several advantages [5]:

- **Targeted Scalability:** In a monolithic system, an unexpected spike in traffic for one specific feature (e.g., the payment gateway) requires scaling the entire application, leading to resource wastage. Microservices allow infrastructure teams to scale only the specific services experiencing high demand, optimizing cloud resource consumption and costs.
- **Fault Isolation and Resilience:** Because microservices are loosely coupled, a critical bug, memory leak, or crash in one service does not inherently cascade and bring down the entire platform. The application can handle partial failures gracefully, degrading functionality while keeping core operations online.
- **Technological Freedom:** Teams are not locked into a single technology stack. Since microservices communicate via standardized network protocols, developers can select the most appropriate programming language, framework, and database for each specific task. This flexibility allowed the Smart Travel project to implement and compare different Java frameworks (Spring Boot and Quarkus) within the same ecosystem.
- **Agility and CI/CD:** Smaller codebases are easier to understand, test, and deploy. This structure naturally complements DevOps practices, enabling CI/CD pipelines to push frequent updates to production.

2.1.3 Challenges and Limitations

Despite its significant advantages, adopting a microservices architecture introduces a high degree of distributed system complexity. Some of the key challenges include:

- **Operational Overhead and Monitoring:** Managing dozens of independently running services is inherently more complex than managing a single monolith. It requires sophisticated container orchestration platforms (such as Kubernetes or OKD) and centralized observability tools for distributed tracing, log aggregation, and metric monitoring to identify where a failure occurred across the network.
- **Network Latency and Congestion:** Replacing in-memory function calls with remote network requests introduces latency. If a single user action requires a long chain of inter-service API calls, the accumulated network overhead can severely degrade system performance.

- **Data Consistency:** Because each service maintains its own database, achieving data consistency across the system becomes a major challenge. Traditional `ACID` transactions that span multiple tables are no longer feasible. Instead, developers must rely on complex distributed transaction patterns and embrace *eventual consistency*.

2.2 Imperative vs. Reactive paradigm

The choice of programming paradigm fundamentally dictates how an application manages system resources, handles concurrency, and scales under load. In the context of modern Java microservices, understanding the underlying thread management of both approaches is crucial for evaluating their suitability for high-concurrency, I/O-bound applications.

2.2.1 The Imperative Model

Traditional applications use blocking I/O and an imperative (sequential) execution model. In a web application context - such as a traditional Spring MVC application - this paradigm relies on the *Thread-per-Request* concurrency model [6].
When a new HTTP request arrives, the server allocates a dedicated thread from a pre-configured thread pool to handle the entire lifecycle of that request. If the application needs to interact with an external resource, such as querying a database or calling a third-party API, the thread initiates the I/O operation and then completely halts execution (blocks) until it receives a response.
While this model is straightforward to write, debug, and reason about, it is highly inefficient for I/O-bound microservices. During the waiting period, the blocked thread consumes valuable memory and CPU cycles via context switching, doing absolutely no useful work. Under sudden spikes in traffic, the thread pool can quickly become exhausted. Once all threads are blocked waiting for slow external services, the application becomes unresponsive to new user requests, severely limiting scalability.

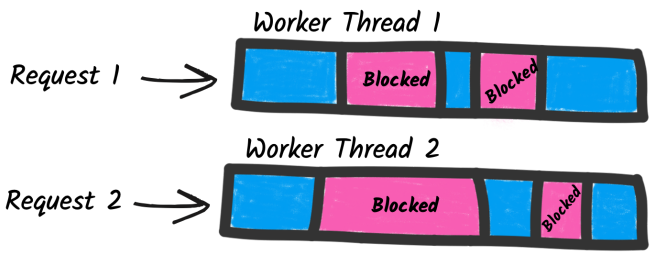

Figure 2.2: Thread-per-Request model in an imperative application. *Source: [7]*

2.2.2 The Reactive Model

The reactive model was designed to build non-blocking applications that are highly responsive, resilient, and resource-efficient. Instead of the Thread-per-Request model, reactive frameworks utilize an *Event Loop* architecture. In this model, a very small number of threads known as `event loop threads` continuously loop to process incoming requests.
When an I/O operation is required, the thread does not block. Instead, it delegates the task to a non-blocking I/O handler, registers a callback, and immediately returns to the event queue to process the next incoming HTTP request. Once the external database or API finishes its processing, it pushes an event back to the event loop, which then resumes the original request and sends the response to the client [7].

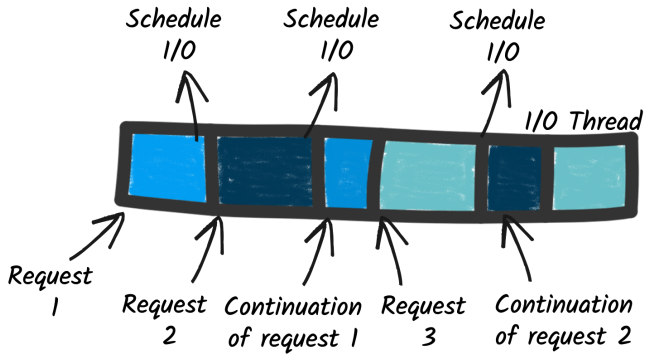

Figure 2.3: Reactive model in a non-blocking application. *Source: [7]*

The key advantages of the reactive model include:

- **Resource Efficiency:** By eliminating blocked threads, the system can handle thousands of concurrent connections using a fraction of the memory and CPU.

- **Data as Streams:** Data is not pulled sequentially but pushed asynchronously. Frameworks like Spring WebFlux and Quarkus Mutiny provide rich APIs to compose and transform these data streams without blocking the main execution flow.

The primary advantage of the reactive concurrency model is its resource efficiency and scalability. By ensuring that threads are never blocked waiting for external I/O operations, the system can sustain thousands of concurrent connections using a fraction of the memory and CPU required by a traditional imperative system. This architectural shift makes the reactive model particularly well-suited for I/O-bound

microservices, allowing them to remain highly responsive and maintain stable performance even under massive, sudden surges in network traffic.
However, this model introduces significant complexities regarding development and operational debugging. The most critical is the inclusion of a blocking operation — such as a legacy JDBC database query, a synchronous HTTP call, or a heavy CPU-bound computation — directly on the event loop. Because the system relies on a small pool of threads, blocking even a single one can immediately bottleneck the application. Furthermore, the non-linear execution flow fragments traditional stack traces. When an exception occurs, the stack trace often only reflects the point of failure within the asynchronous callback, losing the historical context of the original user request and making root-cause analysis considerably more difficult than in the linear imperative model.

2.3 Event-Driven Architectures

As microservices ecosystems expand, the method by which these independent services communicate becomes a critical architectural decision. While traditional web architectures rely heavily on synchronous, point-to-point interactions, the inherent limitations of this approach at scale have led to the widespread adoption of *Event-Driven Architectures* (EDA) [8].

2.3.1 Synchronous vs. Asynchronous Communication

In a synchronous communication model, typically implemented via HTTP and REST APIs, a service sends a request to another service and blocks its own execution until it receives a direct response. While conceptually simple and easy to trace, this tightly coupled approach creates cascading bottlenecks. If the receiving service is slow, overwhelmed, or offline, the calling service is forced to wait, consuming resources and potentially causing system-wide performance degradation [9].
On the other hand, asynchronous communication decouples the sender from the receiver. In an Event-Driven Architecture, services communicate by producing and consuming *events* - lightweight messages that represent a significant change in state or a specific occurrence within the business domain (e.g., "user registered" or "order placed"). The service producing the event simply broadcasts it and immediately resumes its own operations, entirely unaware of who will consume the event or when it will be processed [8].

2.3.2 The Role of Message Brokers

To facilitate this asynchronous exchange without requiring services to know each other's network locations or IP addresses, EDA relies on a centralized intermediary

known as a message broker [10].
This architecture generally relies on three primary components:

- **Producers (Publishers):** The microservices that generate events when a specific business action occurs. Their only responsibility is to format the event data and safely push it to the broker.

- **Message Brokers:** The middleware infrastructure that receives, stores, and routes the events. The broker ensures reliable message delivery, managing queues or topics so that events are safely held even if the intended downstream recipients are currently offline or busy.

- **Consumers (Subscribers):** The microservices that continuously listen to the broker for specific types of events. Upon receiving an event, the consumer processes the data and executes its own independent business logic.

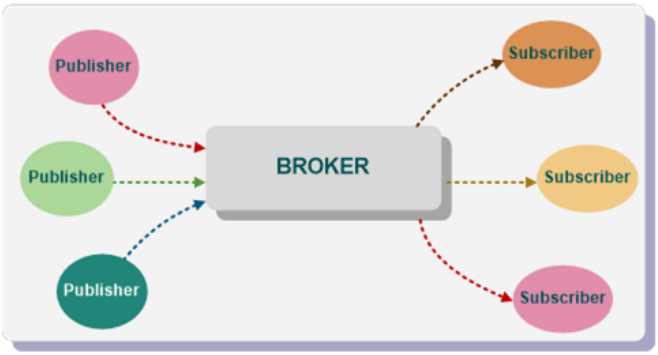

Figure 2.4: Event-Driven Architecture

This publish/subscribe (Pub/Sub) model allows multiple different consumers to react to a single event simultaneously and independently.

2.3.3 Benefits and Challenges

Implementing an Event-Driven Architecture provides different advantages. The decoupling of services enables exceptional scalability, as individual components can be scaled up or down based on their specific event processing loads. Furthermore, it significantly enhances overall system resilience: if a consumer service crashes, the message broker simply retains the pending events in a queue until the service is restored, guaranteeing no data loss during the outage [9].

However, this paradigm introduces unique complexities regarding distributed data. Because communication is fundamentally asynchronous, achieving immediate transactional data consistency across the entire platform is impossible. Instead, developers must design applications around the concept of *eventual consistency*, where the system state reconciles over time. Additionally, tracking the flow of a single user action across multiple asynchronously triggered microservices requires sophisticated distributed tracing mechanisms and centralized logging [10].

2.4 Backend Technologies

Java has been the cornerstone of enterprise backend development for decades, favored for its robustness, mature ecosystem, and platform independence. However, as the industry transitioned from monolithic architectures running on heavy application servers to containerized microservices orchestrated by Kubernetes, traditional Java frameworks faced significant challenges. They often suffered from slow startup times and high memory footprints, characteristics misaligned with the rapid scaling and high-density deployment needs of cloud-native environments [11].
To address these challenges, the Java ecosystem has evolved significantly. During the development of this thesis' project, two distinct frameworks were selected to build the backend microservices: the established Spring Boot and the modern cloud-native Quarkus.

2.4.1 Spring Boot: The Industry Standard

Introduced by Pivotal Software (now VMware Tanzu) in 2014, Spring Boot revolutionized Java enterprise development. It was designed to simplify the bootstrapping and development of new Spring applications by adhering to the *convention over configuration* principle. Before Spring Boot, developers spent hours configuring XML files and managing complex application server deployments. Spring Boot eliminated this friction by providing auto-configuration, opinionated dependency management, and embedded web servers (such as Tomcat or Netty), allowing developers to package an entire microservice into a single, easily deployable `fat JAR` [12].
Today, Spring Boot is the de facto standard for building Java microservices. It can count on an enormous ecosystem, with out-of-the-box integrations for almost every conceivable database, message broker, and cloud provider. While its traditional roots lie in the synchronous, imperative programming model (*Spring MVC*), the framework has actively modernized. With the introduction of *Spring WebFlux*, it now fully supports the reactive, event-loop programming paradigm discussed in Section 2.2, enabling high concurrency without the overhead of thread-per-request blocking.

2.4.2 Quarkus: The Cloud-Native Framework

While Spring Boot adapted to the cloud era, Quarkus was born entirely within it. Created by Red Hat and released in 2019, Quarkus describes itself as a "Kubernetes Native Java framework tailored for GraalVM and HotSpot". Its primary mission is to optimize Java for containerized and serverless environments, where fast startup times (measured in milliseconds) and minimal memory consumption are critical for reducing cloud infrastructure costs and enabling rapid auto-scaling [11].

Quarkus achieves this extreme efficiency by fundamentally changing the Java build process. Traditional frameworks like Spring heavily rely on reflection, dynamic class loading, and classpath scanning at runtime (processes that consume significant memory and slow down the boot time). Quarkus, instead, utilizes a *compile-time boot* technique. It shifts the vast majority of framework initialization and configuration parsing to the build phase (*Ahead-Of-Time* processing).

Furthermore, Quarkus is built from the ground up on a reactive core powered by Eclipse Vert.x [13]. This unified architecture allows developers to seamlessly blend standard imperative code and non-blocking reactive code within the exact same application, dynamically optimizing the underlying execution thread model without requiring the developer to switch between entirely different API stacks.

2.4.3 The Rationale for Comparison

The decision to utilize both Spring Boot and Quarkus within the Smart Travel platform was driven by the desire to evaluate the state of the art in Java microservices. Implementing identical business logic - such as reactive database queries and asynchronous RabbitMQ event publishing/consuming - using both frameworks provides a valuable lens through which to assess their respective developer experiences, syntactic clarity, and ease of configuration. While Spring Boot offers the safety and familiarity of a massive, mature ecosystem, Quarkus promises a leaner, faster, and easier approach to cloud-native development [14].

2.5 Frontend Technologies

Just as backend architectures have evolved to handle greater scale and complexity, frontend development has undergone a massive transformation. The traditional web paradigm, where the server renders and delivers completely new HTML pages for every user interaction, has largely been superseded by the *Single Page Application* (SPA) model. This shift was driven by the need for more dynamic, responsive, and app-like user experiences that could not be achieved with full-page reloads. To implement this modern frontend architecture, the *Angular* framework was selected

for the project, leveraging its powerful component-based structure and native support for reactive programming with the *RxJS* library.

2.5.1 Single Page Application Architecture

Historically, web applications were built as Multi-Page Applications (MPAs). In an MPA, every time a user clicks a link or submits a form, the browser discards the current page, sends a request to the server, waits for the server to process the logic, and then downloads an entirely new HTML document to render. While this approach is straightforward and naturally optimized for *Search Engine Optimization* (SEO), it is highly inefficient for complex interactive applications. The constant full-page reloads lead to latency, wasted bandwidth, and a disjointed user experience [15].
A Single Page Application flips this paradigm. As the name suggests, an SPA loads only a single foundational HTML web page upon the user's initial visit. This initial payload includes all the necessary static assets - such as HTML scaffolding, CSS stylesheets, and the core JavaScript application bundle.
Once the application is loaded into the browser, the page is never reloaded again during the user's session. Instead, when the user interacts with the application (e.g., searching for a flight or filtering hotel results), JavaScript intercepts the action and dynamically rewrites the current DOM (*Document Object Model*). If new data is required, the SPA makes asynchronous requests (via `AJAX` or the `Fetch API`) to backend APIs. The server responds with lightweight JSON data rather than heavy HTML markup. The frontend then processes this data and injects it directly into the existing page structure [15].
This architecture provides several significant advantages:

- **Fluid User Experience:** By eliminating the disruptive "white flash" of page reloads, SPAs provide a continuous, fluid experience that closely mimics a native desktop or mobile application.
- **High Performance and Speed:** After the initial load, subsequent interactions are nearly instantaneous because only raw data is transmitted over the network, drastically reducing bandwidth consumption and server load.
- **Clear Separation of Concerns:** The SPA architecture perfectly complements a microservices backend. The frontend becomes a completely independent application that only cares about consuming data via standardized APIs, allowing frontend and backend teams to develop, deploy, and scale their codebases independently.

Despite its benefits, the SPA model introduces challenges, particularly regarding the initial loading time - since the browser must download a large JavaScript bundle

upfront - and the complexity of managing client-side routing and browser history. To address these complexities, robust frontend frameworks are required.

2.5.2 The Angular Framework

To implement the SPA architecture for the Smart Travel platform, Angular was selected. Maintained by Google, Angular is a comprehensive, enterprise-grade framework specifically designed for building scalable single-page applications [16]. Unlike lightweight libraries that focus solely on rendering views (such as *React*), Angular provides a highly opinionated, all-encompassing toolset. It enforces a strict architectural pattern, requiring developers to write code using TypeScript—a superset of JavaScript that introduces static typing, interfaces, and advanced object-oriented features. This ensures greater code reliability and maintainability in large-scale projects.

Angular applications are built upon a *Component-based architecture*. The UI is divided into encapsulated, reusable building blocks called components. Each component strictly separates its concerns into three parts: a TypeScript class that manages the business logic and state, an HTML template that defines the view, and a CSS file scoped exclusively to that component's styling.

Furthermore, Angular features a powerful *Dependency Injection* (DI) system, which allows components to easily request shared services (like an HTTP client to fetch catalog data) without needing to instantiate them manually, promoting highly modular and testable code [16].

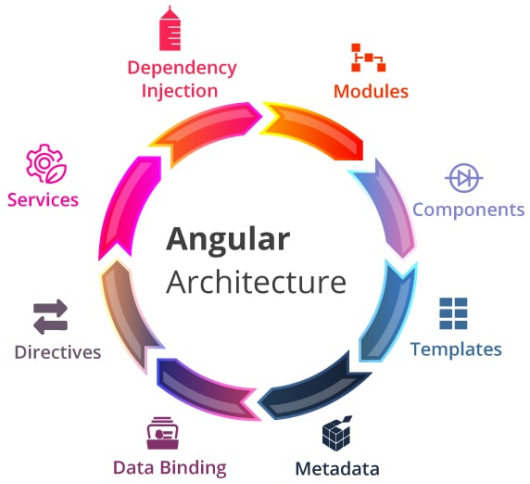

Figure 2.5: Angular Architecture. *Source: [17]*

2.5.3 Frontend Reactive Programming

Because SPAs heavily rely on asynchronous events — such as user inputs, delayed API responses, and routing changes — managing this complex flow of data is a major challenge. To handle this, Angular natively integrates the RxJS library.
RxJS brings the reactive programming paradigm (discussed in Section 2.2) directly into the browser. It introduces the concept of *Observables*, which represent continuous streams of asynchronous data or events over time [18].
Instead of relying on traditional JavaScript `Promises`, which can only handle a single future value and cannot be cancelled, Observables can emit multiple values over time and can be closed or cancelled at any moment. For example, when performing a search query, the Angular HTTP client returns an Observable. Using RxJS operators (like `map`, `filter`, or `debounceTime`), the frontend can gracefully transform the incoming data, ignore rapid repeated clicks from the user, and seamlessly update the UI state without resorting to convoluted, heavily nested callback functions [18]. By leveraging RxJS, the frontend achieves the same non-blocking, event-driven resilience as the reactive backend microservices.

2.6 Database Technologies

As modern applications demand greater flexibility, performance, and the ability to handle massive volumes of unstructured or semi-structured data, traditional *Relational Database Management Systems* (RDBMS) often face significant limitations. The rigid, tabular schema of SQL databases requires careful upfront planning and costly migrations when data requirements change. Furthermore, scaling a relational database is typically restricted to vertical scaling (adding more CPU or RAM to a single server), which eventually hits a hardware ceiling [19]. To address these challenges, *NoSQL* databases emerged as a powerful alternative, offering distributed architectures and highly adaptable data models.

2.6.1 NoSQL Concepts and Classifications

NoSQL databases depart from the fixed table-and-row model, prioritizing horizontal scalability, high availability, and agility over strict ACID compliance. Instead of forcing data into rigid relational structures, NoSQL systems allow data to be stored in formats that more closely align with the objects used in modern application code [20].
NoSQL databases are generally categorized into four primary models:

- **Key-Value Stores:** The simplest NoSQL model, where data is stored as a collection of unique keys and associated values (e.g., Redis). They are highly optimized for fast lookups and caching.

- **Wide-Column Stores:** Data is organized into tables, rows, and dynamic columns, allowing each row to have different columns. They excel at handling massive, predictable query workloads (e.g., Cassandra).
- **Graph Databases:** Designed to store and traverse relationships between entities using nodes and edges (e.g., Neo4j). They are ideal for social networks, fraud detection, or recommendation engines.
- **Document Databases:** Data is stored in flexible, JSON-like documents. Each document contains pairs of fields and values, which can include nested arrays and objects. This is the category to which MongoDB belongs [19].

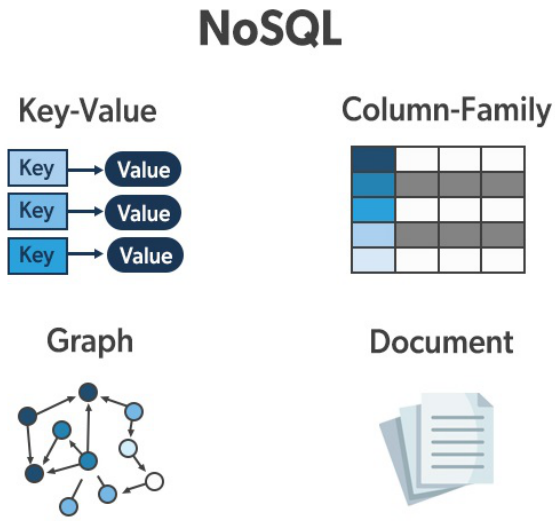

Figure 2.6: NoSQL Database Types

2.6.2 MongoDB Architecture and Flexible Schema

The Smart Travel application leverages MongoDB as the primary data store. MongoDB is the leading NoSQL document database, natively designed to handle modern application data with superior flexibility and scalability .
Instead of tables and rows, MongoDB organizes data into *Collections* and *Documents*. Under the hood, documents are stored in a binary format called `BSON` (Binary JSON), which extends standard JSON with additional data types like dates, decimals, and raw binary data. This document-oriented architecture natively maps to the object-oriented structures in Java or TypeScript, completely eliminating the need for complex *Object-Relational Mapping* (ORM) layers and expensive SQL `JOIN` operations [20].

A core advantage of MongoDB is its flexible Schema (often referred to as dynamic schema). In a traditional SQL database, adding a new feature—such as introducing a "discount" or "dietary requirements" field to only a subset of travel packages—requires an expensive `ALTER TABLE` migration that can lock the database. In MongoDB, documents within the same collection do not need to have identical structures. A new field can simply be appended to a new document without affecting existing records . This schema flexibility significantly accelerates the development lifecycle and perfectly accommodates the diverse, evolving attributes of different travel products in the Smart Travel catalog.

2.6.3 Scalability and High Availability

Beyond its data model, MongoDB's architecture is built for distributed environments:

- **Replica Sets:** To ensure high availability, MongoDB uses Replica Sets — a cluster of nodes (typically one Primary and multiple Secondaries) that maintain identical copies of the data. If the Primary node fails, the system automatically elects a new Primary, providing seamless failover and redundancy [20].

- **Sharding:** When the data volume exceeds the capacity of a single machine, MongoDB handles horizontal scaling through Sharding. The data is partitioned across multiple servers based on a designated "Shard Key," allowing the database to manage massive datasets without degrading query performance.

While MongoDB offers immense freedom, it still requires careful data modeling. Developers must strategically choose between *embedding* related data (storing it within the same document for faster read operations) and *referencing* it (linking separate documents via IDs to avoid duplicating large amounts of data). This decision is crucial for optimizing query performance and ensuring data integrity, especially in a complex domain like travel where entities such as flights, hotels, and user profiles have intricate relationships.

2.7 API Paradigms: REST vs. GraphQL

In a decoupled architecture where the frontend is separated from the backend microservices, the API serves as the fundamental communication bridge. The design of these APIs dictates how efficiently data is transferred over the network. For the "Smart Travel" platform, evaluating the right API paradigm was essential for optimizing the interaction between the Angular frontend and the Java backend. The two dominant paradigms in modern web development are *REST* and *GraphQL*.

2.7.1 REST (Representational State Transfer)

REST has been the undisputed standard for web APIs for over a decade. Introduced by Roy Fielding in 2000, it is an architectural style rather than a strict protocol. RESTful APIs are built around the concept of "resources" (e.g., users, flights, orders), which are identified by standard URLs. Clients interact with these resources using standard HTTP methods such as `GET` (to retrieve data), `POST` (to create data), `PUT` (to update), and `DELETE` [21].
A well-designed REST API is stateless, meaning each client request must contain all the information the server needs to fulfill it, and the server does not store any client context between requests. This statelessness makes REST highly scalable and cacheable, which is why it remains the default choice for general-purpose microservice communication.
However, as frontend applications (like SPAs) have become more complex, REST has revealed significant limitations regarding data fetching:

- **Over-fetching:** A REST endpoint typically returns a fixed data structure. If the Smart Travel frontend only needs the names and prices of available flights to display a simple list, a `GET /flights` request might still return massive JSON objects containing seating charts, airline details, and baggage rules that the client must download but ultimately discard [22].

- **Under-fetching (the N+1 problem):** If a view requires data from multiple related resources — such as a user's profile and their past order history — the client is often forced to make multiple sequential HTTP requests to different endpoints (e.g., `GET /users/123` followed by `GET /users/123/orders`). This significantly increases network latency and degrades the user experience.

2.7.2 GraphQL: The Client-Driven Alternative

To solve the inefficiencies of over-fetching and under-fetching, Facebook developed GraphQL, open-sourcing it in 2015. Unlike REST, which exposes multiple URLs for different resources, a GraphQL API typically exposes only a single endpoint (e.g., `/graphql`) [23].
Instead of the server dictating the structure of the response, GraphQL transfers control to the client. It is a strictly typed query language that allows the frontend to explicitly ask for exactly the data it needs, and nothing more. The client sends a specific query document in the body of an HTTP `POST` request, and the server responds with a JSON object that exactly mirrors the shape of the requested query. For example, a single GraphQL query can simultaneously fetch a user's name, their last three orders, and the specific hotel names associated with those orders. The backend GraphQL engine acts as an orchestrator, resolving the disparate data

points from various underlying databases or microservices and assembling them into a single, highly optimized response [22].

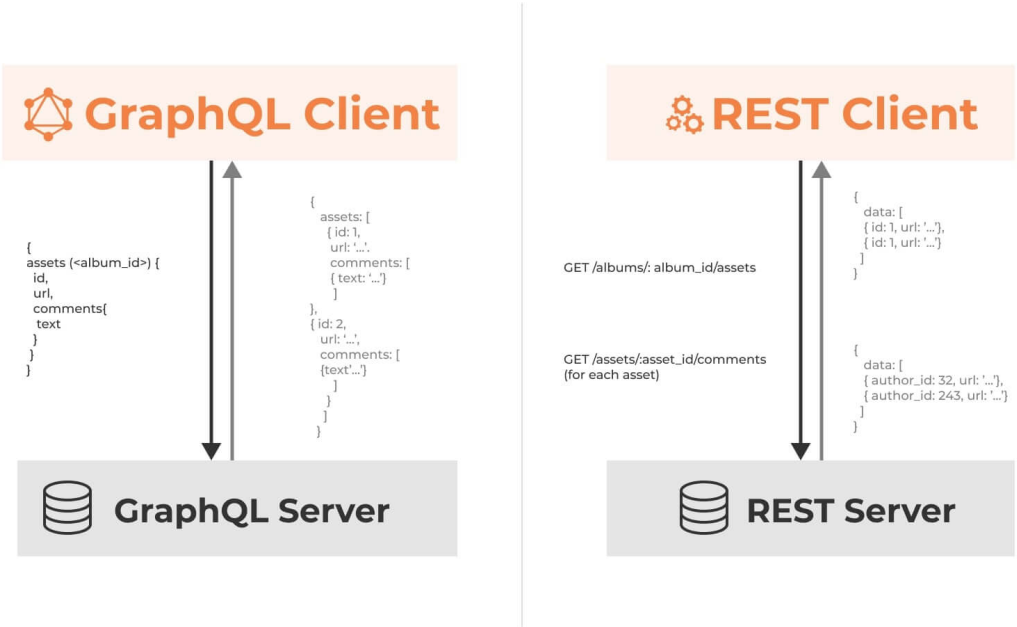

Figure 2.7: REST vs. GraphQL

2.7.3 Choosing the Right Paradigm

While GraphQL offers unparalleled efficiency for frontend data fetching, it introduces significant complexity on the backend. Developing a GraphQL server requires defining a rigorous schema, implementing complex resolver functions, and carefully managing database queries to prevent malicious or overly deep queries from exhausting server resources. Furthermore, traditional HTTP caching mechanisms are much harder to implement with GraphQL than with REST [23].
In the context of modern platforms like Smart Travel, the two paradigms are often not mutually exclusive. REST remains highly effective for server-to-server microservice communication and simple CRUD (Create, Read, Update, Delete) operations, while GraphQL is increasingly favored as an API Gateway layer to serve perfectly tailored data payloads to frontend applications.

Chapter 3

System Architecture

3.1 Requirements Analysis: Functional and non-functional requirements of the Smart Travel platform.
3.2 High-Level Architecture: Decomposing the system into microservices (Users, Catalog, Orders, Notifications).
3.3 Data Modeling: Designing the MongoDB collections and handling flexible schemas.
3.4 Communication Patterns: * 3.4.1 Synchronous communication (REST/GraphQL)
3.4.2 Asynchronous communication (RabbitMQ events)

Chapter 4

Application Design and Implementation

4.1 Backend Development: * 4.1.1 Implementing the reactive services with Spring Boot
4.1.2 Implementing the reactive services with Quarkus
4.2 Frontend Development: Building the user interface, state management with RxJS, and connecting to the backend.
4.3 Core Workflows:
4.3.1 The catalog search and filtering mechanism
4.3.2 The payment integration flow
4.3.3 The event-driven notification system

Chapter 5

Infrastructure Automation and Cloud Deployment

5.1 The DevOps Culture: Bridging the gap between development and operations.
5.2 Continuous Integration & Continuous Delivery (CI/CD): Automating the build, test, and containerization process using Jenkins.
5.3 Container Orchestration with Kubernetes/OKD: * 5.3.1 OKD Cluster overview
5.3.2 Workload management: Pods and Deployments
5.3.3 Configuration and Security: ConfigMaps and Secrets
5.3.4 Networking: Services and Route exposure

Chapter 6

Conclusion

6.1 Technology Evaluation: Spring Boot vs. Quarkus: Comparing developer experience, startup times, resource consumption, and reactive capabilities.
6.2 Architectural Evaluation: Assessing the scalability, resilience, and maintainability of the implemented microservices and event-driven patterns.
6.3 Testing and Validation: Ensuring the functional requirements of the Smart Travel application were met.

Bibliography

[1] Muskaan Goyal and Pranav Bhasin. *Moving from monolithic to microservices architecture for multi-agent systems*. 2025. URL: https://arxiv.org/pdf/2505.07838 (cit. on p. 1).

[2] Amazon Web Services (AWS). *What are Microservices?* 2025. URL: https://aws.amazon.com/microservices/ (cit. on p. 5).

[3] Microsoft Learn. *Microservices Architecture Style*. 2025. URL: https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices (cit. on p. 6).

[4] CoderCo. *Monoliths vs Microservices: A Guide to Choosing the Right Architecture for Your Application*. 2023. URL: https://blog.coderco.io/p/monoliths-vs-microservices-a-guide (cit. on p. 6).

[5] Eyitayo Olaigbe. *Microservices Architecture: Building Resilient, Scalable Applications in the Cloud Era*. 2024. URL: https://www.tayoolaigbe.com/articles/microservices-architecture (cit. on p. 7).

[6] Ray Dhiraj. *Spring WebFlux Reactive REST API*. 2023. URL: https://www.devglan.com/spring-boot/spring-webflux-reactive-rest-api (cit. on p. 8).

[7] Quarkus Official Documentation. *Quarkus Reactive Architecture*. 2024. URL: https://quarkus.io/guides/quarkus-reactive-architecture (cit. on pp. 8, 9).

[8] IBM. *What Is Event-Driven Architecture?* 2024. URL: https://www.ibm.com/topics/event-driven-architecture (cit. on p. 10).

[9] Microsoft Learn. *Asynchronous message-based communication*. 2022. URL: https://learn.microsoft.com/en-us/dotnet/architecture/microservices/architect-microservice-container-applications/asynchronous-message-based-communication (cit. on pp. 10, 11).

[10] Red Hat. *Event-driven architecture: Understanding the essential benefits*. 2021. URL: https://www.redhat.com/en/blog/event-driven-architecture-essentials (cit. on pp. 11, 12).

[11] Rich Sharples / Red Hat. *Bringing Java into the Kubernetes-native future with Quarkus*. 2020. URL: https://www.redhat.com/en/blog/bringing-java-kubernetes-native-future-quarkus (cit. on pp. 12, 13).

[12] Spring.io. *Microservices with Spring*. 2025. URL: https://spring.io/microservices/ (cit. on p. 12).

[13] Eclipse Vert.x. *Vert.x*. URL: https://vertx.io (cit. on p. 13).

[14] Hitesh Umaletiya / Brilworks. *Spring Boot vs Quarkus vs Micronaut Performance and Use Case Guide*. 2025. URL: https://www.brilworks.com/blog/spring-boot-vs-quarkus-vs-micronaut/ (cit. on p. 13).

[15] Adobe Experience Cloud Team. *Single-page applications (SPAs) — what they are and how they work*. 2023. URL: https://business.adobe.com/blog/basics/learn-the-benefits-of-single-page-apps-spa (cit. on p. 14).

[16] Angular Team. *What is Angular? - Official Documentation*. 2025. URL: https://angular.dev/overview (cit. on p. 15).

[17] Besant Technologies. *What is Angular?* URL: https://www.besanttechnologies.com/what-is-angular (cit. on p. 15).

[18] ReactiveX. *ReactiveX: The Observer pattern done right*. 2025. URL: https://reactivex.io/ (cit. on p. 16).

[19] Zoumana Keita. *NoSQL Databases: What Every Data Scientist Needs to Know*. 2023. URL: https://www.datacamp.com/blog/nosql-databases-what-every-data-scientist-needs-to-know (cit. on pp. 16, 17).

[20] Roshni Verma. *MongoDB Architecture: A Comprehensive Guide*. 2024. URL: https://medium.com/@roshniverma021/mongodb-architecture-a-comprehensive-guide-c72799347b22 (cit. on pp. 16–18).

[21] Amazon Web Services (AWS). *What is a RESTful API?* 2025. URL: https://aws.amazon.com/what-is/restful-api/ (cit. on p. 19).

[22] IBM. *GraphQL vs. REST: What's the Difference?* 2023. URL: https://www.ibm.com/topics/graphql-vs-rest (cit. on pp. 19, 20).

[23] Apollo GraphQL. *Introduction to GraphQL*. 2024. URL: https://www.apollographql.com/docs/intro/ (cit. on pp. 19, 20).